Name : Shivraj Vaibhav Jagtap
Roll No : EN24107060
Class : SY - AI\&DS
Subject : Python for Visual AI

Assignment 1:

Convert a color image to grayscale with different methods mentioned below and compare their performances (SSIM, Entropy, PSNR, etc.)

1. Average Method (Arithmetic Mean)
2. Lightness Method
3. Luminosity (Perceptual) Method
4. Luma Method

Step 1: Importing the necessary python libraries

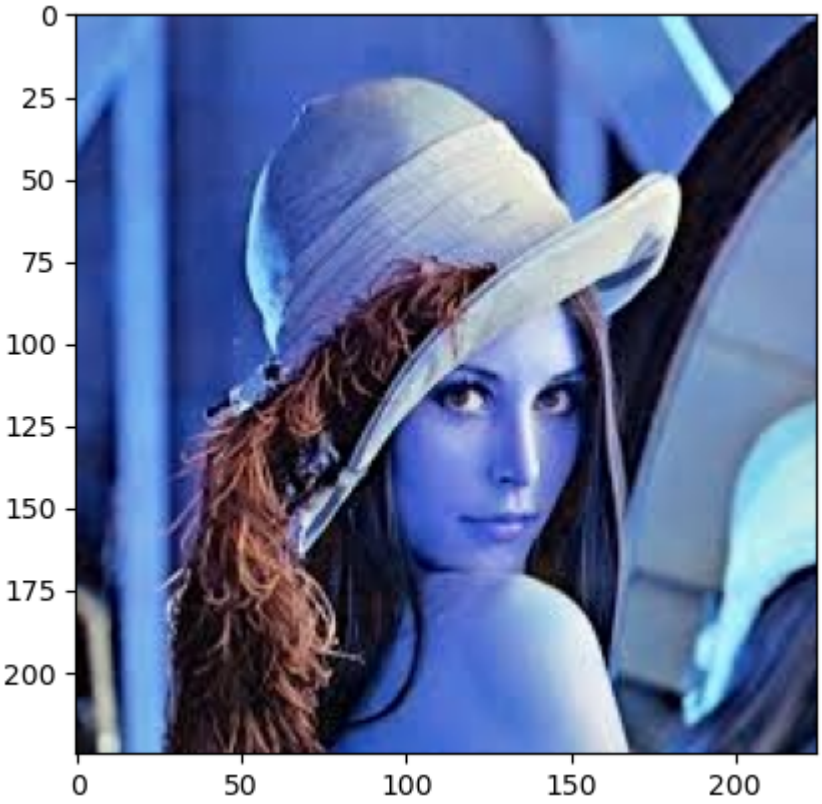
In [1]:
```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt
import latexify
import numpy as np
from math import log10, sqrt
import skimage
from skimage.metrics import structural_similarity as ssim
```

Step 2: Loading and reading the image that is to be converted into grayscale

In [2]:
```
img=cv.imread("lena.jpeg")
```

In [3]:
```
cv.imwrite("img.jpg",img)
img1= cv.imread("img.jpg")
plt.imshow(img1)
```

Out[3]:
```
<matplotlib.image.AxesImage at 0x26ef311c830>
```



Step 3: Converting the image datatype into float

We convert the image to a float datatype to maintain precision during grays

In [4]:
```
img_f=img.astype(np.float64)
r,g,b=cv.split(img_f)
cv.imwrite("img_f.jpg",img_f)
```

Out[4]: True

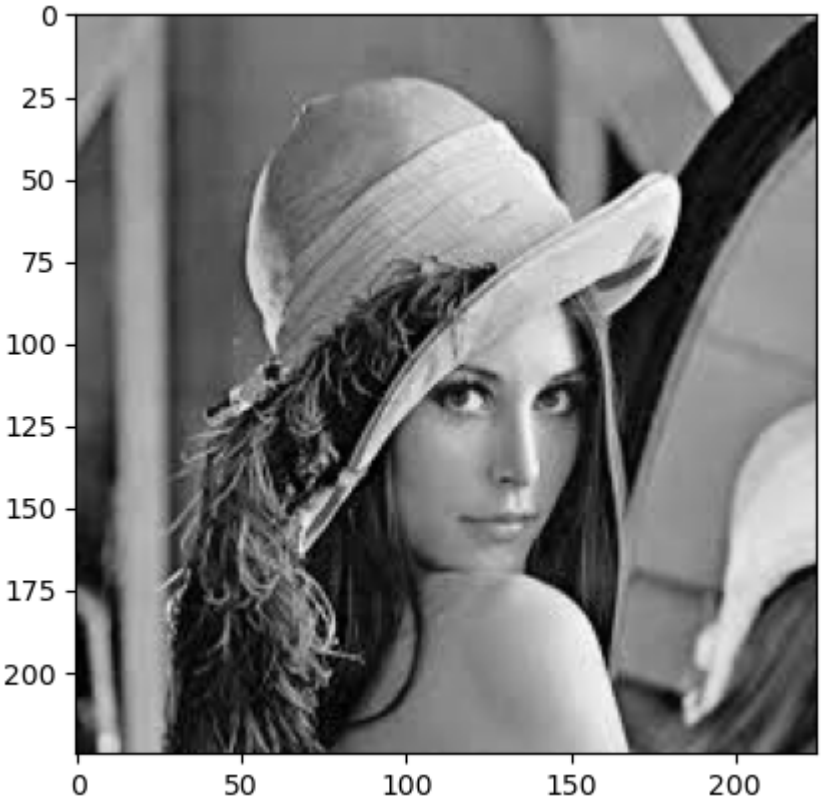
Method 1: Average Method

- This method works by taking the arithmetic mean of the Red, Green, and I

$$Gray = \frac{R+G+B}{3}$$

In [5]:
```
img_avg=(r+g+b)/3
cv.imwrite("avg_img.jpg",img_avg)
img2= cv.imread("avg_img.jpg")
plt.imshow(img2)
```

Out[5]: <matplotlib.image.AxesImage at 0x26ef331afd0>

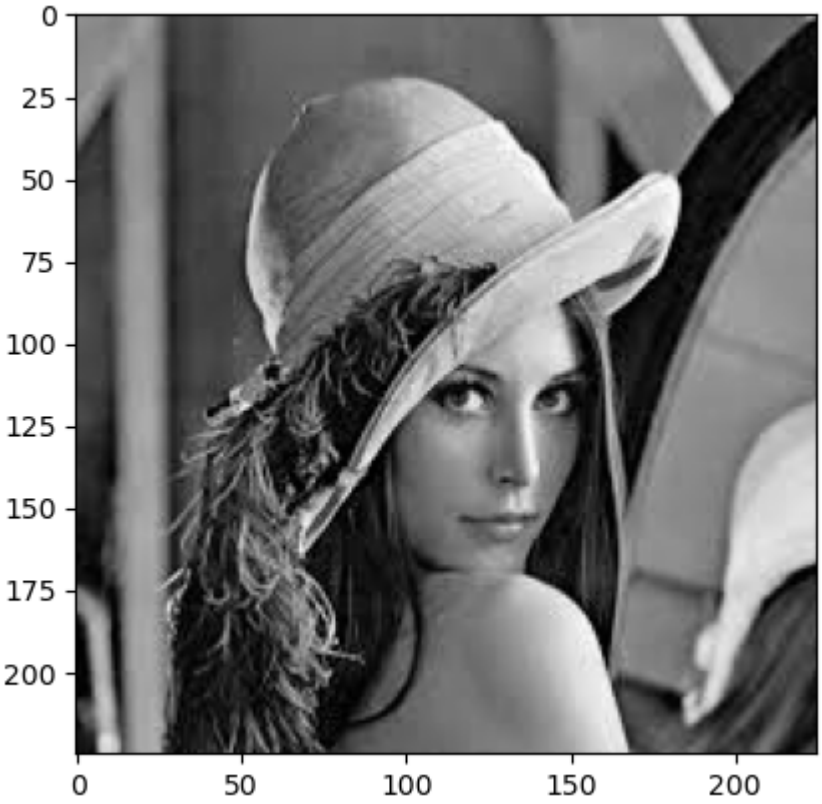


Method 2: Luminosity Method

- The Luminosity Method is a more sophisticated approach to grayscale conv
- Unlike the average method, it accounts for the human eye's varying sensitiv

$$Gray = 0.29912R + 0.587G + 0.114B$$

In [6]:
```
luminosity=(0.29912*r)+(0.587*g)+(0.114*b)
cv.imwrite("lumi_img.jpg",luminosity)
img3= cv.imread("lumi_img.jpg")
lumi_img=plt.imshow(img3)
```

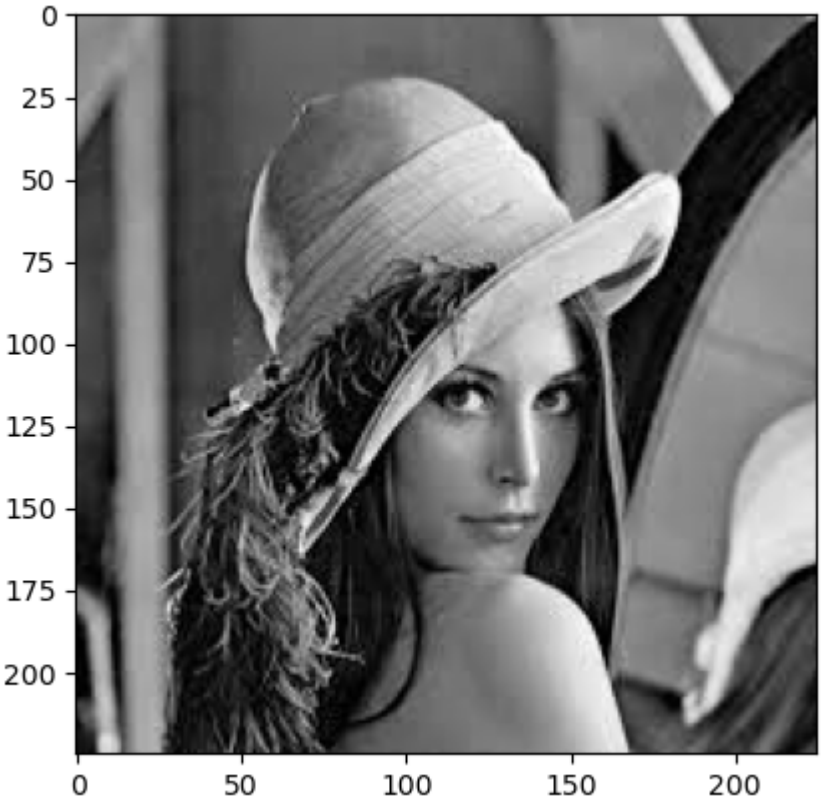


Method 3: Luma Method

- The Luma Method is the industry standard for high-definition television (H
- It further refines the weights to match the color space of modern monitors.

$$Gray = 0.2126R + 0.715G + 0.0722B$$

In [7]:
```
luma=(0.2126*r)+(0.715*g)+(0.0722*b)
cv.imwrite("luma_img.jpg",luma)
img4= cv.imread("luma_img.jpg")
luma_img=plt.imshow(img4)
```

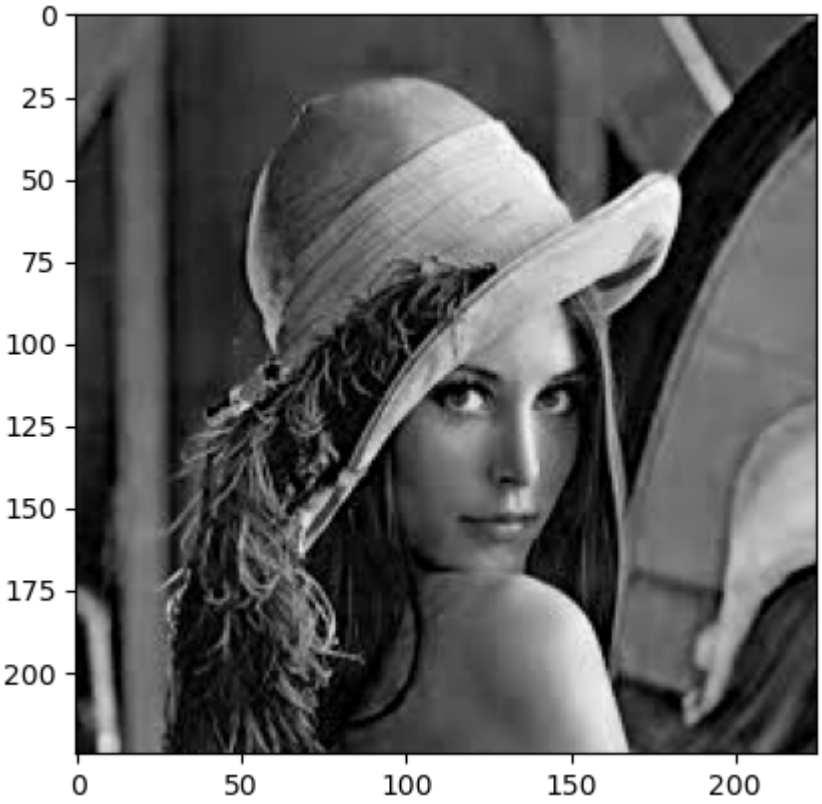


Method 4: Lightness Method

- The Lightness Method calculates the gray value by averaging the most pro

$$Gray = \frac{\max(R, G, B) + \min(R, G, B)}{2}$$

In [8]:
```
lightness=(cv.max(r,g,b)+cv.min(r,g,b))/2
cv.imwrite("light_img.jpg",lightness)
img5= cv.imread("light_img.jpg")
light_img=plt.imshow(img5)
```



Step 4 : Comparing the factors like PSNR, SSIM & Entropy

PSNR

Peak Signal-to-Noise Ratio (PSNR) is a standard metric used to measure the pixel-level similarity between a reconstructed or processed image and its original version. In the context of grayscale conversion or compression, a higher PSNR value generally indicates that the resulting image is of better quality, as it suggests a lower level of error or "noise" introduced during the transformation.

$$PSNR = 20 \cdot \log_{10}\left(\frac{MAX_I^2}{MSE}\right)$$

Where:

MAX_I	is the maximum possible pixel value (1.0 for float, 255 for uint8).
MSE	is the Mean Squared Error between the two images.

In [9]:
```
def PSNR1 (I, I1):

    mse = np.mean((I - I1)**2)

    if mse == 0:
        return float('inf') #only possible if both the images are identic
    max_pixel = 255.0
```

```
    psnr = 20 * log10(max_pixel / sqrt(mse))
    return psnr

print("PSNR of Original and Average:\t",PSNR1(img1,img2))
print("PSNR of Original and Lumi:\t",PSNR1(img1,img3))
print("PSNR of Original and Luma:\t",PSNR1(img1,img4))
print("PSNR of Original and Light:\t",PSNR1(img1,img5))
```

```
PSNR of Original and Average:    28.460763173661817
PSNR of Original and Lumi:       29.209054048639498
PSNR of Original and Luma:       29.44779533532541
PSNR of Original and Light:      30.00185261749431
```

Entropy

Entropy measures the amount of information or "disorder" in an image. A high entropy value indicates an image with a high degree of detail, texture, or random noise, while low entropy suggests a simpler image with many uniform areas (like a solid background).

$$H = -\sum_{i=0}^{L-1} P_i \log_2(P_i)$$

Where:

H is the Shannon Entropy.
P_i is the probability of occurrence of intensity i.
L is the total number of intensity levels (e.g., 256).

In [10]:
```
img_e1=skimage.measure.shannon_entropy(img_f)
print("The Entropy of Original Image is:\t\t",img_e1)
img_e2=skimage.measure.shannon_entropy(img2)
print("The Entropy of Grayscale (Avg) Image is:\t",img_e2)
img_e3=skimage.measure.shannon_entropy(img3)
print("The Entropy of Grayscale (Lumi) Image is:\t",img_e3)
img_e4=skimage.measure.shannon_entropy(img4)
print("The Entropy of Grayscale (Luma) Image is:\t",img_e4)
img_e5=skimage.measure.shannon_entropy(img5)
print("The Entropy of Grayscale (Light) Image is:\t",img_e5)
```

```
The Entropy of Original Image is:                7.929229744165612
The Entropy of Grayscale (Avg) Image is:         7.768202169517702
The Entropy of Grayscale (Lumi) Image is:        7.775448096054106
The Entropy of Grayscale (Luma) Image is:        7.786761847083904
The Entropy of Grayscale (Light) Image is:       7.628042573412785
```

Structural Similarity Index (SSIM)

SSIM is a perception-based model that considers image degradation as perceived change in structural information. While PSNR only looks at pixel errors, SSIM measures three key features: **Luminosity**, **Contrast**, and **Structure**.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

Where:

μ_x, μ_y	The average pixel intensities of images x and y.
σ_x, σ_y	The variance (contrast) of images x and y.
σ_{xy}	The covariance (cross-correlation) of images x and y.
c_1, c_2	Constants to prevent division by zero.

In [11]:
```
dr=255.0

score1=ssim(img_f,img2,channel_axis=2,data_range=dr)
score2=ssim(img_f,img3,channel_axis=2,data_range=dr)
score3=ssim(img_f,img4,channel_axis=2,data_range=dr)
score4=ssim(img_f,img5,channel_axis=2,data_range=dr)
print("SSIM Value of avg_img is:\t",score1)
print("SSIM Value of lumi_img is:\t",score2)
print("SSIM Value of luma_img is:\t",score3)
print("SSIM Value of light_img is:\t",score4)
```

```
SSIM Value of avg_img is:        0.9105548320175866
SSIM Value of lumi_img is:       0.9090205978911126
SSIM Value of luma_img is:       0.9064031139446703
SSIM Value of light_img is:      0.8523435248368157
```

Conclusion

- Luminosity and Luma methods provide the best grayscale quality.
- Average and Lightness methods are simple but less accurate.
- Higher SSIM and PSNR values indicate better preservation of image struct
- Luma method is preferred in video processing, while Luminosity is widely u

Here we will consider **Luminosity Method** for **Grayscale Conversion** of the image